Monte Carlo Based Parameter Estimation of the Li–Rinzel Calcium Oscillation Model Using RMSE & Parametric KL Divergence

GROUP-9

INTRODUCTION

The Li–Rinzel model is a **reduced nonlinear dynamical system** consisting of a coupled set of two first-order ordinary differential equations. It exhibits rich oscillatory behavior and is widely used as a **benchmark nonlinear ODE system for studying parameter sensitivity, numerical simulation, and inverse regression problems**. The qualitative and quantitative behavior of the model is highly dependent on a small set of unknown control parameters, making it an ideal testbed for **nonlinear parameter estimation and optimization techniques**.

In this project, we address the **inverse problem of identifying unknown Li–Rinzel model parameters from time-series data**. The governing ODE system is solved numerically using a **finite-difference Euler scheme**, and the simulated solution is mapped onto the experimental sampling grid using **interpolation** in order to ensure consistent temporal comparison.

Parameter estimation is performed using a **Monte Carlo–based stochastic global search strategy**, which allows continuous exploration of the multidimensional parameter space without imposing deterministic search bias. The model–data mismatch is quantified using a **composite objective function** constructed from:

- **Root Mean Square Error (RMSE)** in the time domain,
- **Parametric Kullback–Leibler (KL) divergence**, computed via maximum likelihood–based probability density fitting,
- **Dominant frequency mismatch**, extracted using Fast Fourier Transform (FFT).

AREAS OF CONTRIBUTIONS

S.NO	NAME	TOPIC	CONTRIBUTIONS	REMARKS
1	G.SAI CHARAN	DATA & TIME LOADING + PREPROCESSING	Created dynamic plots to visualize signal changes	Helped in understanding time-based patterns
2	P. VIVEK GAUTHAM	KL DIVERGENCE AND MONTE CARLO SIMULATION	Wrote Algorithm for monte carlo simulation	Found best parameters minimizing kl and frequency difference
3	HARSHA VARDHAN	LI-RINZEL MODEL SIMULATION	Completed code	Obtained graph for the same
4	KRISH	VISUALIZATIONS	Converted signals to frequency domain	Obtained frequency spectrum for analysis

LIST OF ALL NEW PYTHON FUNCTIONS USED:

- We used "**matplotlib.animation**" to create live, updating visualizations of the calcium dynamics model. This function let us animate the simulation frame by frame, so I could clearly show how the parameters affected Ca^{2+} signals over time. It made the results much easier to interpret and present visually.

- **"scipy.signal.find_peaks"** is a function I used in my project to automatically detect peaks in calcium signals. It identifies local maxima based on height, distance, or prominence, which helped me extract meaningful spike patterns from noisy Ca^{2+} data.

- **"MinMaxScaler"** from "**sklearn.preprocessing**" scales data to a fixed range, usually 0 to 1. In a project context, it helps normalize calcium signals so different simulations can be compared on the same scale. This avoids bias from large-value features and makes analysis more consistent.
- **"np.errstate"** is a NumPy context manager that lets me temporarily control how numericalwarnings behave, such as divide-by-zero or invalid operations. In my project, I used it to safelyhandle edge cases during calculations without interrupting the simulation.

- **"scipy.fft.rff"t** computes the fast Fourier transform for real-valued signals. I used it toanalyze the frequency components of calcium dynamics and understand oscillation patterns moreclearly.

- we used "**scipy.fft.rfftfreq**" to generate the corresponding frequency values for the FFToutput. It helped me map the transformed data to real frequencies and interpret which oscillationmodes were present in the calcium signals.

Data Loading & Preprocessing (Contributor – CHARAN)

- Core Logic
  - This block loads the raw calcium-intensity data, aligns it with the experimental timestamps, removes offsets, normalizes the signal, and prepares it for model fitting.

- Data & Time Loading
  - pd.read_excel(...): Loads intensity + time files.
  - df.iloc[:, 3]: Extracts the calcium signal column.
  - **Length alignment:**
  - **Valid-data mask:** removes NaN/∞.
  - **Sort by time:** ensures strictly increasing timestamps.

Time Processing

- Millisecond → Second Conversion
- If

  “max(t)>1000”
- Then ,

  t(sec)= t/1000

Shift time so experiment starts at zero

t(shifted)=t(sec)−t(sec)[0]

Compute Experimental Time Step

Δt(exp)=median(t[i+1]−t[i])

Signal Conditioning

- Baseline Correction
- First 20% of the data = “resting state.

- Baseline value:
- baseline = median($x_{0:0.2N}$)

- Corrected signal:
- $\mathbf{X(corr) = x - baseline}$

Min–Max Normalization

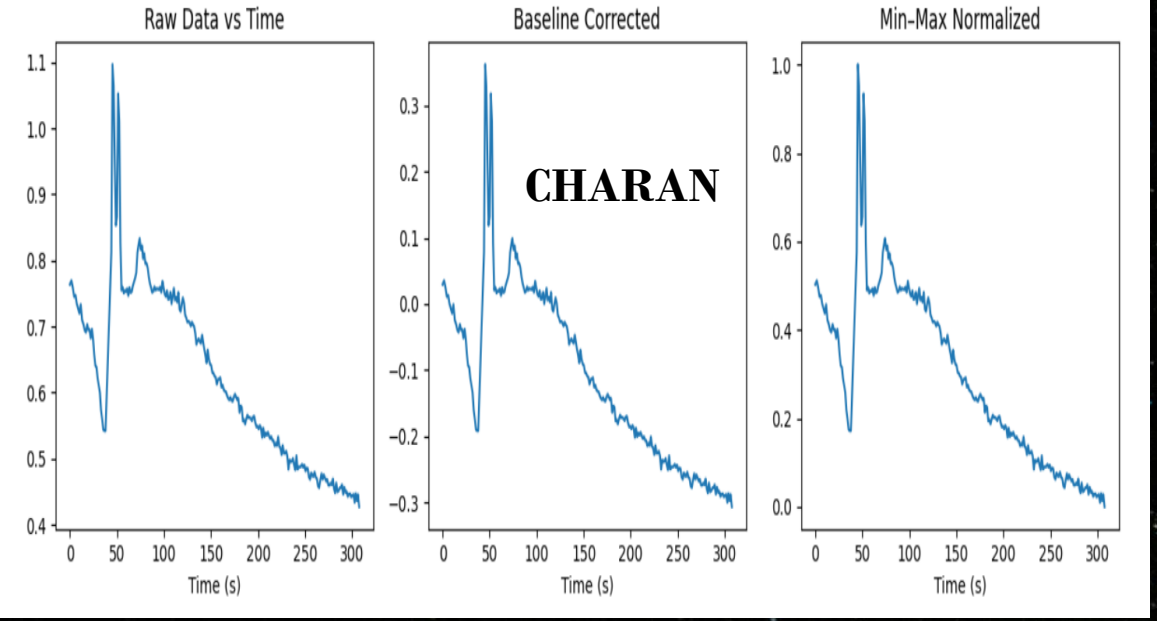
- Using Scikit-Learn's MinMaxScaler:
- $X(norm) = [x(corr) - min(x(corr))] / [max(x(corr)) - min(x(corr))]$
- This scales the data to [0, 1] to match non-dimensional model outputs.

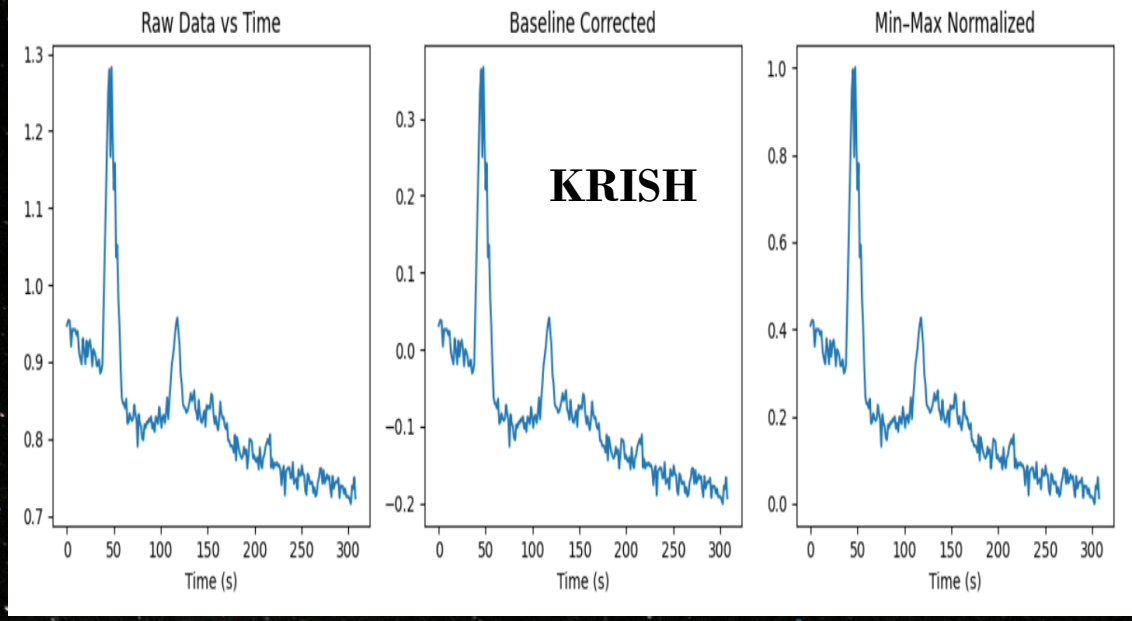
Visual Output (3-Panel Diagnostic Plot)

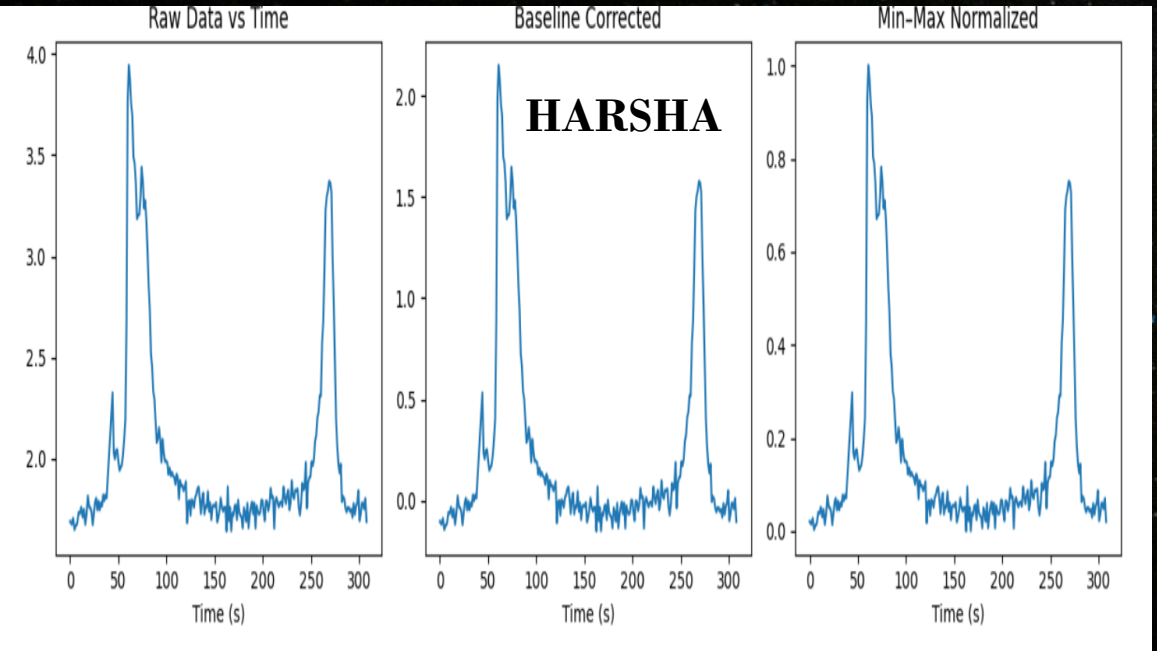
- Raw signal vs time (s)
- Baseline-corrected signal
- Min–Max normalized signal

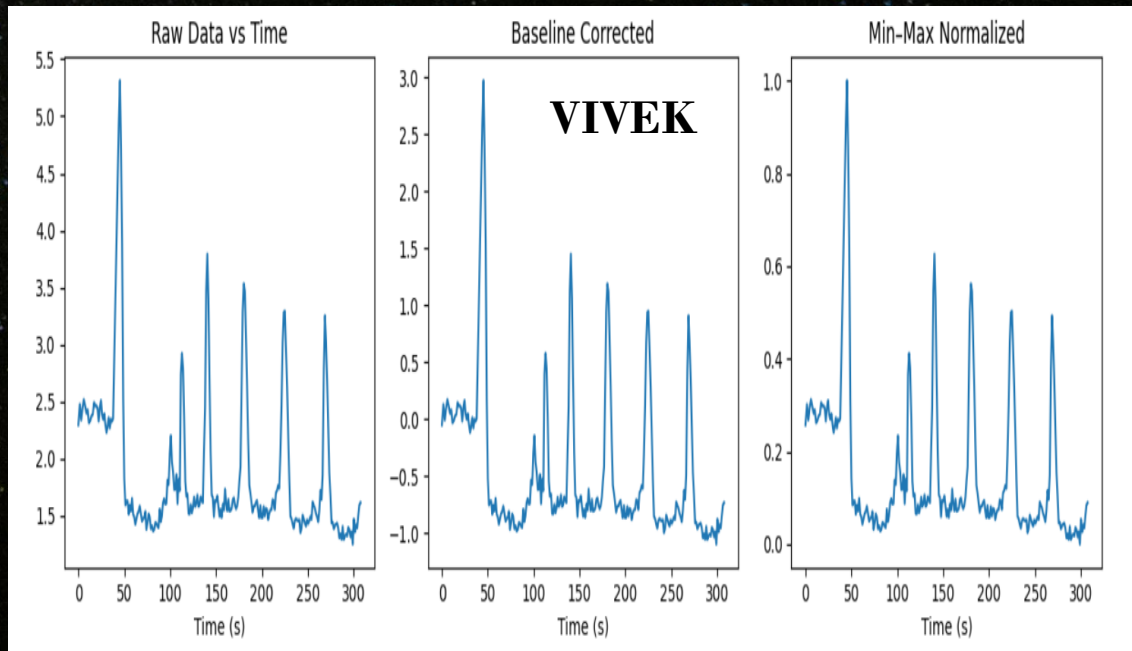
Simulation Readiness

- **$\Delta t(SIM) = \Delta t(exp)$**
- **Ensures the model uses the same time resolution as the experiment.**

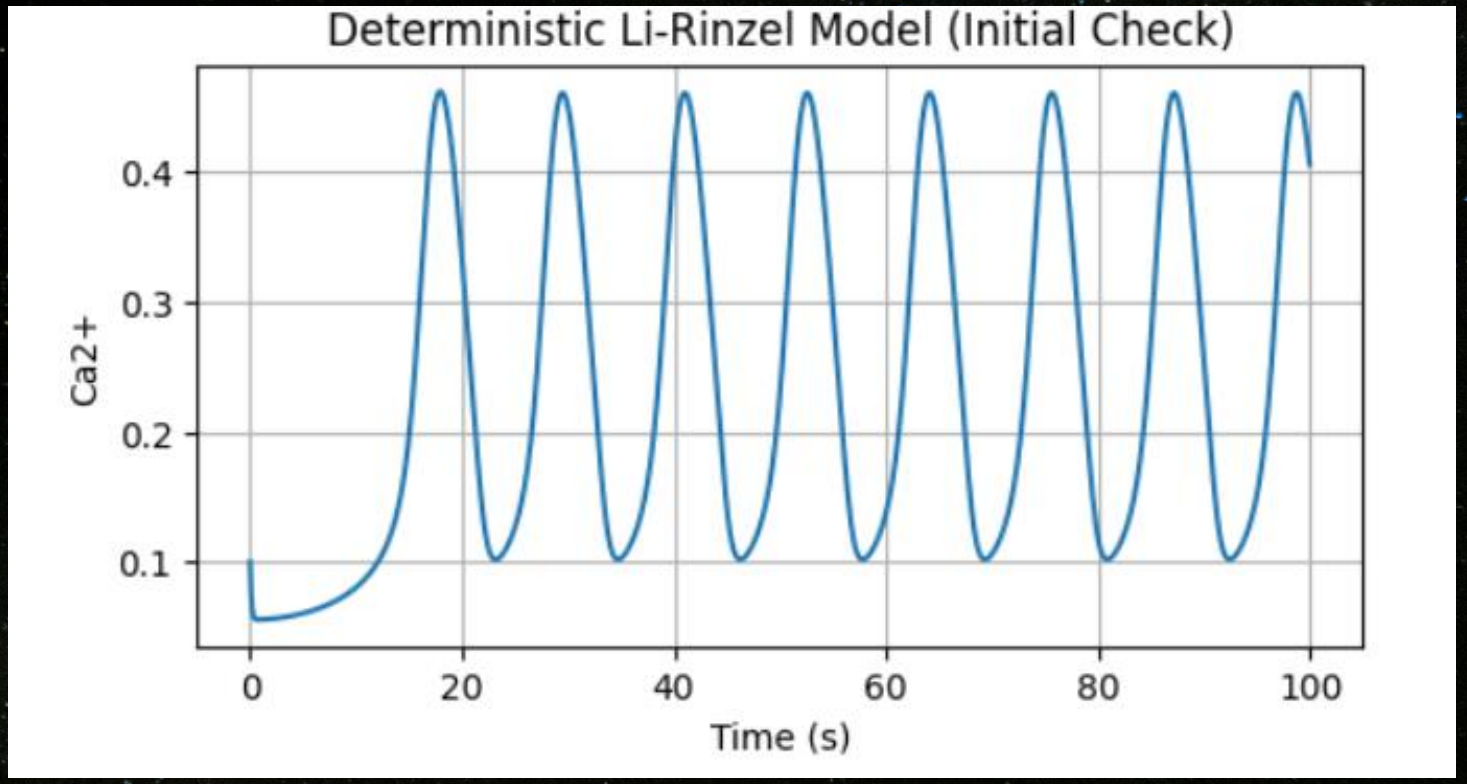

Raw Data vs Time
Baseline Corrected
Min-Max Normalized
CHARAN
Time (s)


Raw Data vs Time
Baseline Corrected
Min-Max Normalized
KRISH
Time (s)


Raw Data vs Time
Baseline Corrected
Min-Max Normalized
HARSHA
Time (s)


Raw Data vs Time
Baseline Corrected
Min-Max Normalized
VIVEK
Time (s)

Li-Rinzel Model Simulation (HARSHA)



- Core Logic
- This block numerically simulates intracellular calcium dynamics using the Li–Rinzel model, integrating calcium concentration (Ca) and the IP_3-receptor inactivation variable (h) using the Euler method.

- **Model Parameters**
- **The model uses standard Li–Rinzel constants:**
- **IP_3 receptor gating:** d_1, d_2, d_3, d_5, a_2
- **Flux strengths:** v_1, v_2, v_3
- **Buffer/ER parameters:** c_0, c_1
- **Initial conditions:**
- $Ca(0) = 0.1, h(0) = 0.1$

Time Grid:

- $t = 0 : dt : t_{\max}$
- Produces discrete time points for Euler integration.

- Calcium Gating Variables:
- IP_3 activation gate
- $p = \frac{IP_3}{IP_3 + d_1}$
- Calcium-dependent inactivation
- $Q_2 = d_2 \frac{IP_3 + d_1}{IP_3 + d_3}$
- Fraction of activated receptors
- $n = \frac{Ca}{Ca + d_5}$

ER Calcium:

- $Ca_{ER} = \frac{c_0 - Ca}{c_1}$

- Inactivation Gate Dynamics (h-variable):
- Steady-state inactivation
- $h_\infty = \frac{Q_2}{Q_2 + Ca}$
- Time constant
- $\tau_h = \frac{1}{a_2(Q_2 + Ca)}$
- ODE for h
- $\frac{dh}{dt} = \frac{h_\infty}{\tau_h}(1 - h) - \frac{1 - h_\infty}{\tau_h} h$

Calcium Fluxes:

- (a) IP_3-dependent channel flux
- $J_{chan} = c_1 v_1 p^3 n^3 h^3 (Ca_{ER} - Ca)$

- b) Passive ER leak
- $J_{leak} = c_1 v_2 (Ca_{ER} - Ca)$

- c) SERCA pump
- $J_{pump} = v_3 \frac{Ca^2}{Ca^2 + k_3^2}$

- Calcium ODE
- $\frac{dCa}{dt} = J_{chan} + J_{leak} - J_{pump}$

Numerical Integration (Euler Method)

- $Ca(t+1) = Ca(t) + dt \cdot dCa/dt$
- $h_{t+1} = h_t + dt \cdot \frac{dh}{dt}$

Parametric KL Divergence + Monte Carlo Optimization (Contributor – VIVEK)

Parametric KL Divergence (MLE-Based)

- Core Logic
- This block compares experimental vs. simulated calcium signals using statistical distance, not pointwise error.
  Both signals are converted into probability distributions, and KL divergence quantifies mismatch.

Gaussian KL Divergence (Analytical Formula)

- Fit Gaussian via MLE
- For each dataset x:
- μ = **mean estimate,**
- σ = **standard deviation estimate**
- (using scipy.stats.norm.fit).

Closed-Form KL Divergence Between Two Gaussians

- For
- $P = \mathcal{N}(\mu_p, \sigma_p^2)$,
- $Q = \mathcal{N}(\mu_q, \sigma_q^2)$
- the KL divergence is:
- $D_{KL}(P \parallel Q) = \ln\left(\frac{\sigma_q}{\sigma_p}\right) + \frac{\sigma_p^2 + (\mu_p - \mu_q)^2}{2\sigma_q^2} - \frac{1}{2}$

Gamma Distribution KL Divergence (Numerical Integration)

- Because Gamma KL has no simple closed form, we:
- Shift data to positive region.
- Fit gamma parameters $k$and θvia MLE:
- Gamma$(x; k; \theta)$
- Generate PDFs $p(x)$,$q(x)$on a grid.
- Compute KL via numerical integration:
- $D_{KL}(P \parallel Q) = \int p(x) \ln\frac{p(x)}{q(x)}\, dx \approx \sum p(x_i) \ln\frac{p(x_i)}{q(x_i)} \Delta x$
- Robust clipping + normalization ensures numerical stability.

- Pure Monte Carlo Search (No Grid Search):

Core Logic:

- Randomly sample Li–Rinzel parameters uniformly in biologically valid ranges, run simulations, compute mismatch metrics, and pick the best fit.

Random Parameter Sampling:

- **Each parameter sampled independently:**
- $k_3 \sim U(0.05, 0.15), v_2 \sim U(0.05, 0.15), d_5 \sim U(0.05, 0.15), a_2 \sim U(0.1, 0.3)$
- **Total simulations:**
- $N = 900$

Simulation → Preprocessing → Alignment:

- Run Li–Rinzel model → simulated Ca(t)
- Min–max normalize using the scaler from preprocessing
- Interpolate simulated data onto experimental time grid:
- $\hat{s}(t_{exp}) = interp(s_{sim}, t_{sim})$

Objective Metrics:

- 1. Frequency Difference
- Compute peak-based oscillation frequency of each trace:
- $f = \frac{1}{\Delta t_{peak}}$
- FreqDiff $= | f_{exp} - f_{sim} |$

RMSE:

- $\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(x_{exp,i} - x_{sim,i}\right)^2}$

Optimization Score:

- The optimization tries to minimize:
- $\text{Score} = FreqDiff + D_{KL}$
- Lower score = better match.

Visualization:

- 1. 2D Monte Carlo RMSE Heatmap
- Shows error landscape over (k3, v2).
- 2. 3D RMSE Surface
- Displays curvature of search space and best-fit point.

Visualization (Contributor – KRISH)

Purpose: Show search landscape, best fit quality, KL behavior, and distribution comparisons.

- **Objective & KL Heatmaps (Monte Carlo Parameter Space):**

- **Core Idea:**
- For each Monte Carlo sample, two quantities are computed:
- **Objective = |Δf| + KL** (frequency mismatch + statistical mismatch)
- **Parametric KL Divergence** (Gaussian-based)
- Both are normalized (0–1) *only for visualization*.

- **Plot Logic**
- Two contour maps are drawn over the parameter plane (k3, v2):.
- **1. Objective Heatmap**
- Uses "tricontourf" to show smooth error landscape.
- Darker regions → **higher mismatch**.
- A white star marks the **best-fit parameter pair**.
- Scatter points show all random samples.

Slide Note:
This heatmap visualizes how Monte Carlo search explores irregular landscape—no convexity assumed.

KL Divergence Heatmap:

- **Same parameter space but color-coded with KL(exp || sim).**
- **Highlights how probabilistic mismatch changes with parameters.**
- **Best-fit point annotated for comparison to Objective map.**

3D Objective Surface:

- Purpose:
- Display curvature and structure of the objective landscape in 3D.
- The shape indicates how difficult the optimization problem is (nonconvex, multimodal).

Core Logic:

- **Construct a triangular surface (plot_trisurf) from random samples.**
- **Z-axis = Normalized Objective.**
- **Helps visualize local minima, smoothness, and ruggedness of search space.**

Slide Highlight:

The shape indicates how difficult the optimization problem is (nonconvex, multimodal).

• Before vs After Parameter Estimation (Requirement 5):

- **Purpose:**
- **Demonstrate improvement due to Monte Carlo estimation.**

Before Plot:

- **Simulated trace using original default Li–Rinzel parameters.**
- **Typically misaligned in amplitude/frequency.**
- **Computes baseline KL_before and RMSE_before.**

After Plot:

- Uses best Monte Carlo parameters.
- Shows substantial improvement in shape, timing, and distribution.
- KL and RMSE visibly lower.
- Slide Highlight:
  This visually proves optimization effectiveness.

Parametric KL Continuity (Gaussian MLE):

Distribution Comparison:

- **Fits Gaussian distributions to experimental and best simulated traces via MLE.**
- **Plots PDFs p(x) and q(x).**
- **Shaded regions show overlap vs mismatch.**

KL Integrand Plot:

- **Plots the function:**
- $p(x) \ln \frac{p(x)}{q(x)}$
- **Key properties:**
- **Shows where signals differ most in probability space.**
- **Confirms smooth continuity of integrand — required for stable KL estimation.**

Time-Resolved KL Divergence :

Purpose

- **Track mismatch over time using sliding windows.**

Logic

- **Sliding window of 100 samples, step size 20.**
- **Compute KL for each window.**
- **Scatter plot with KL encoded in color.**

Slide Insight:
Identifies time intervals where model deviates from experimental signal (good for error localization).

- **Gamma Distribution KL Visualization**

Purpose:

- Show KL behavior under non-Gaussian assumptions.

Plot Logic:

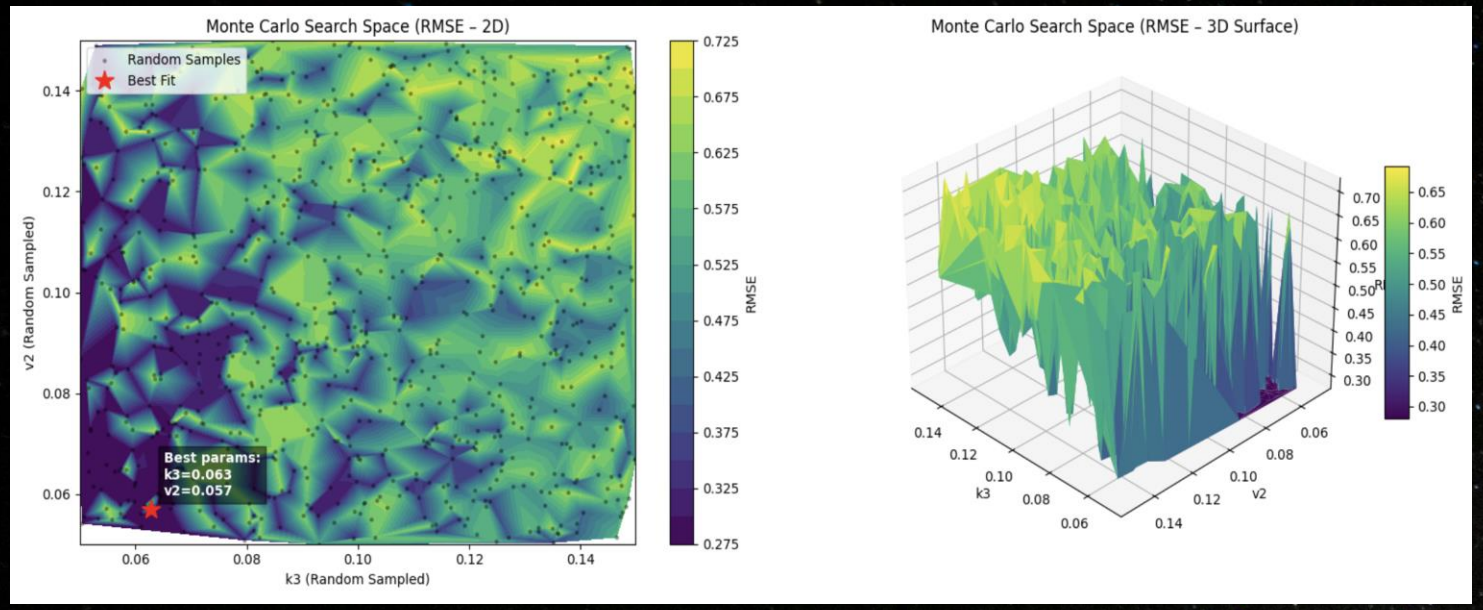
- Fit Gamma distributions to both signals:
  - Shape (k), Scale (θ) via MLE.
- Plot PDFs and compute numerical KL:
- $D_{KL}(P \parallel Q) \approx \sum p\,(x_i) \ln \frac{p(x_i)}{q(x_i)}\,\Delta x$

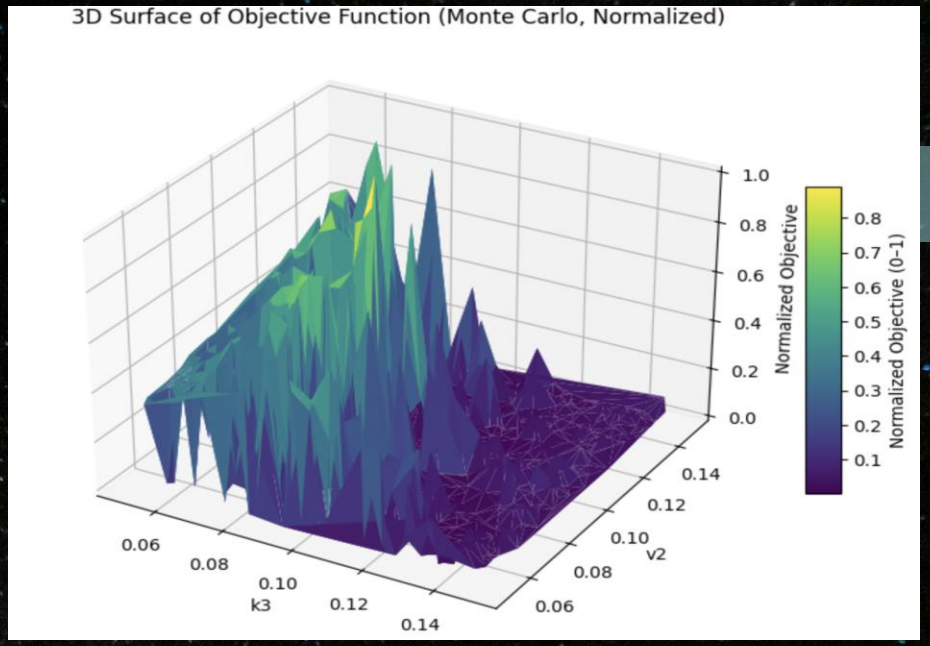
Fallback:

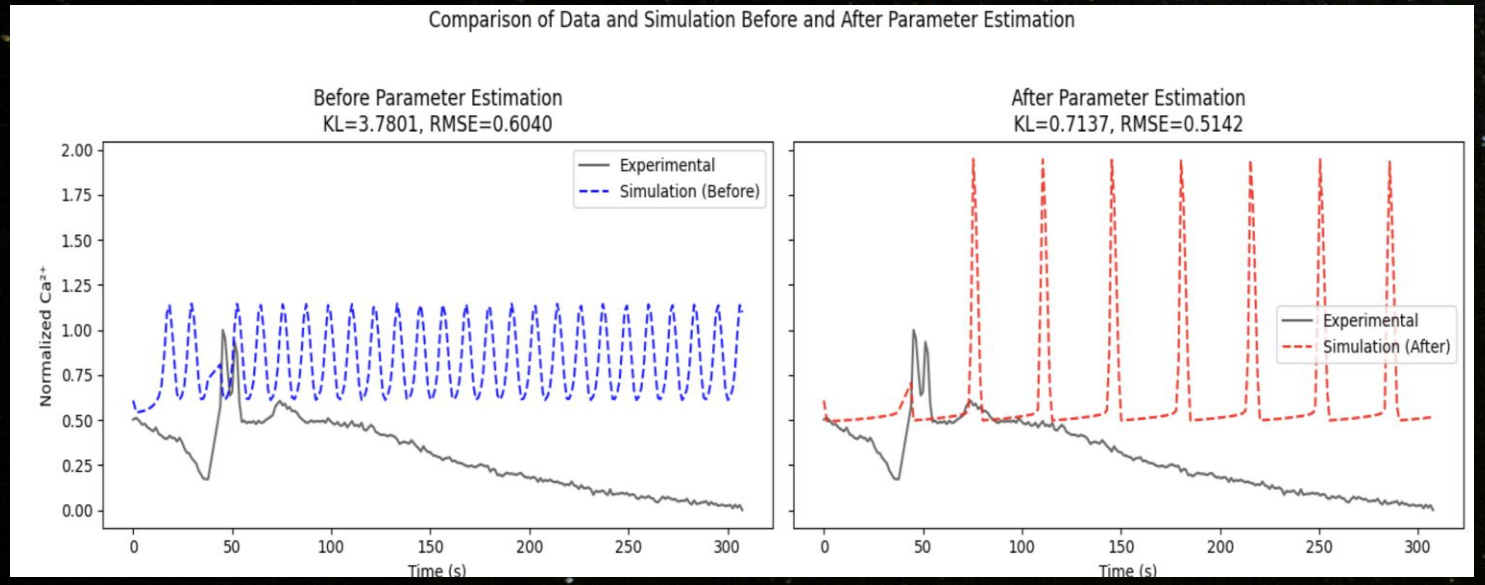
- If Gamma MLE fails (common when signals contain zeros), histogram comparison is shown.

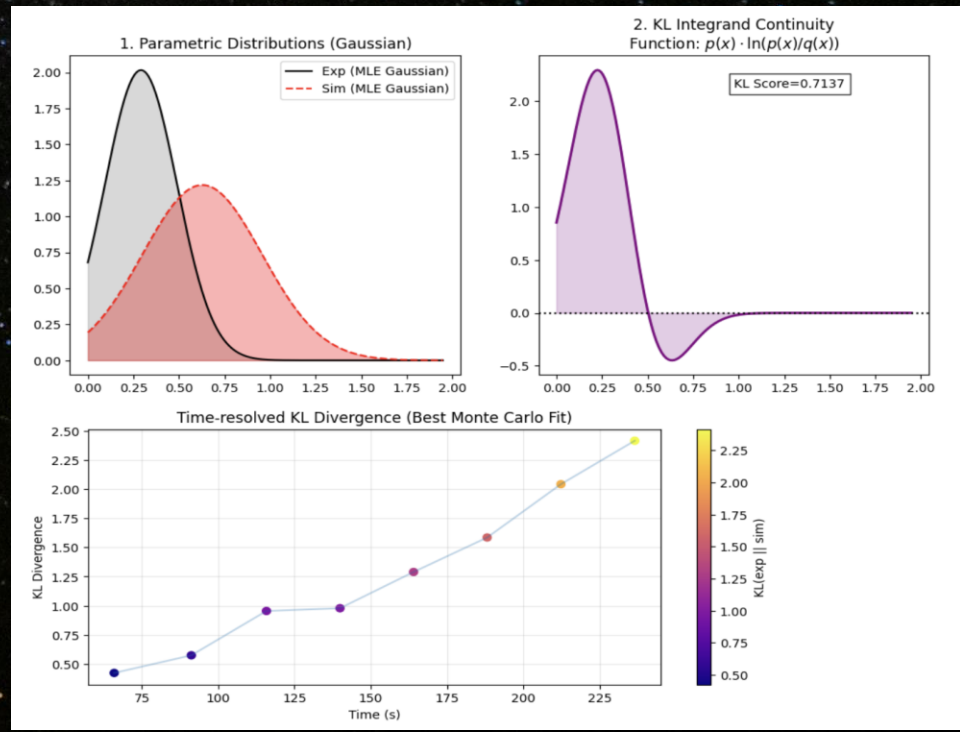
Slide Note:
Gamma fit highlights amplitude asymmetry in non-negative signals like Ca^{2+}.

Column 1 - Charan

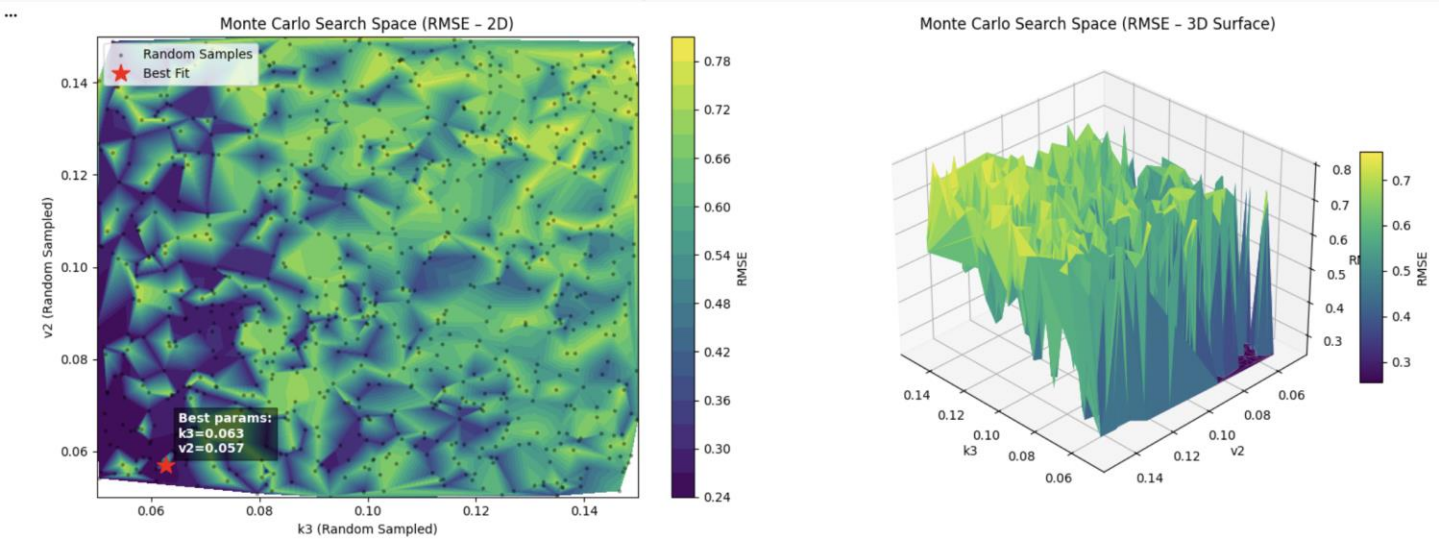


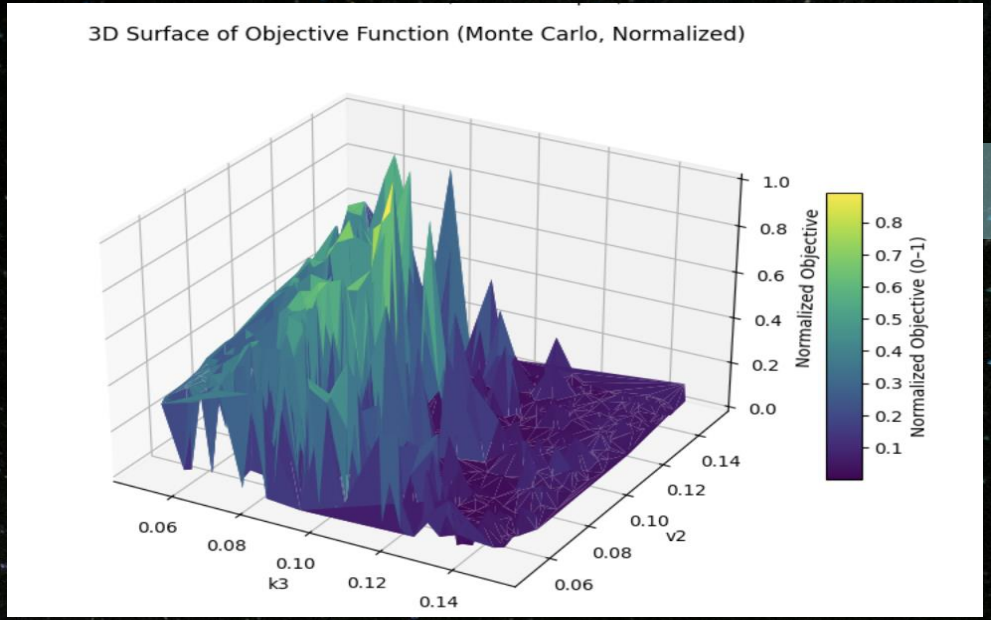


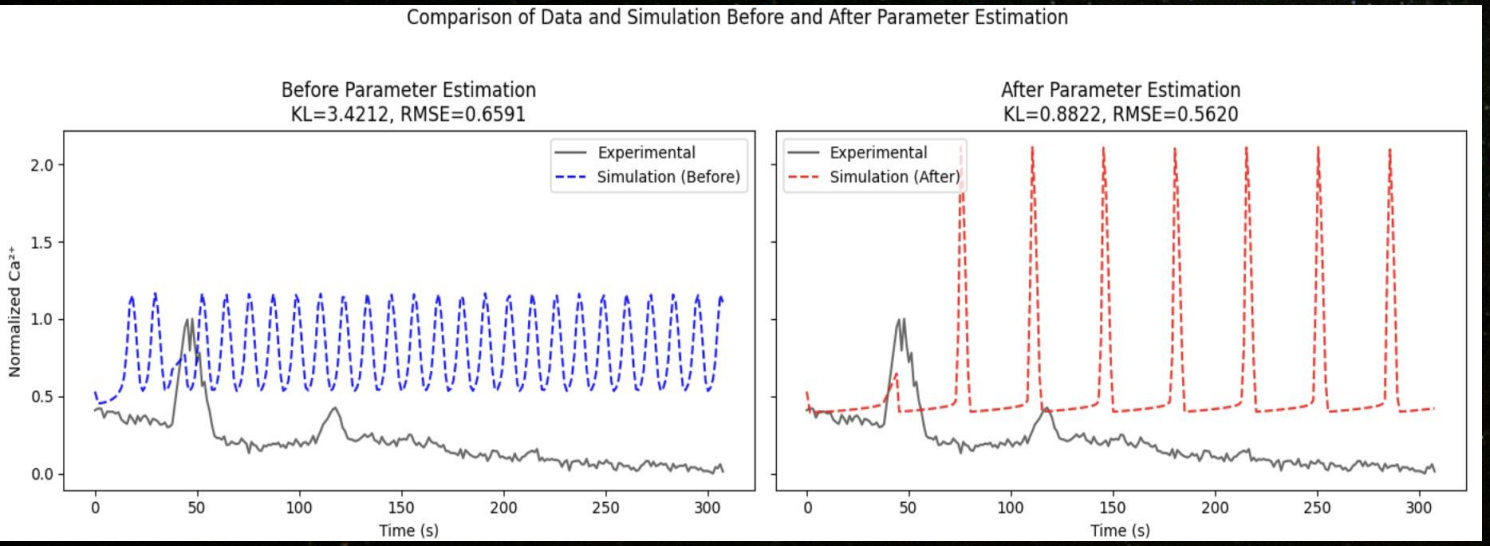


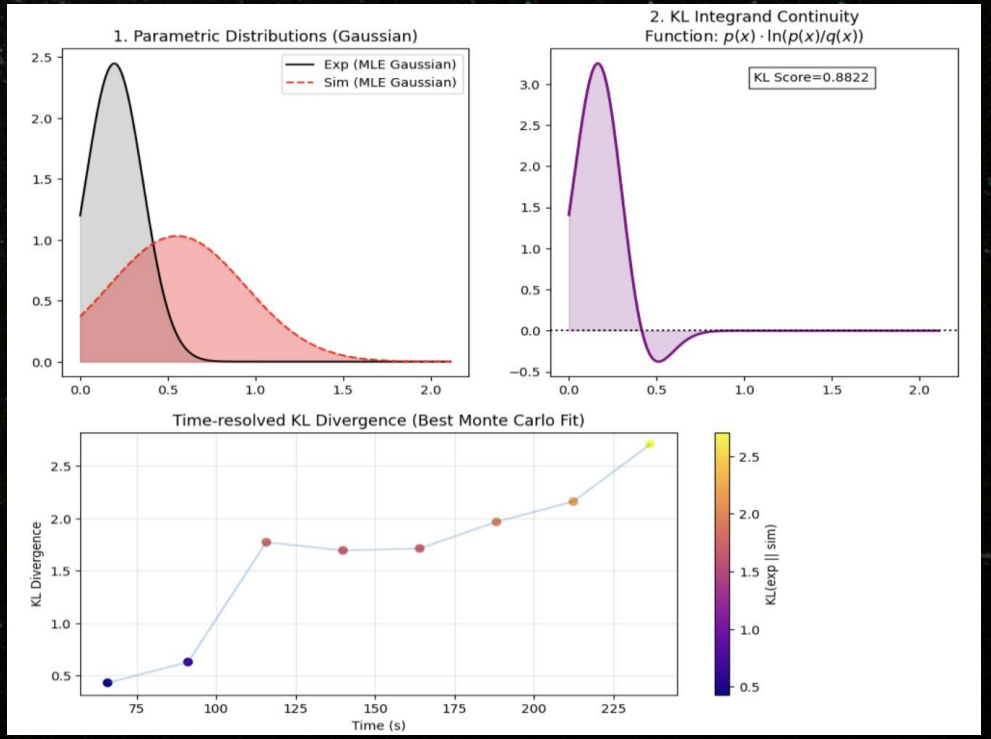


Column 2 – Krish

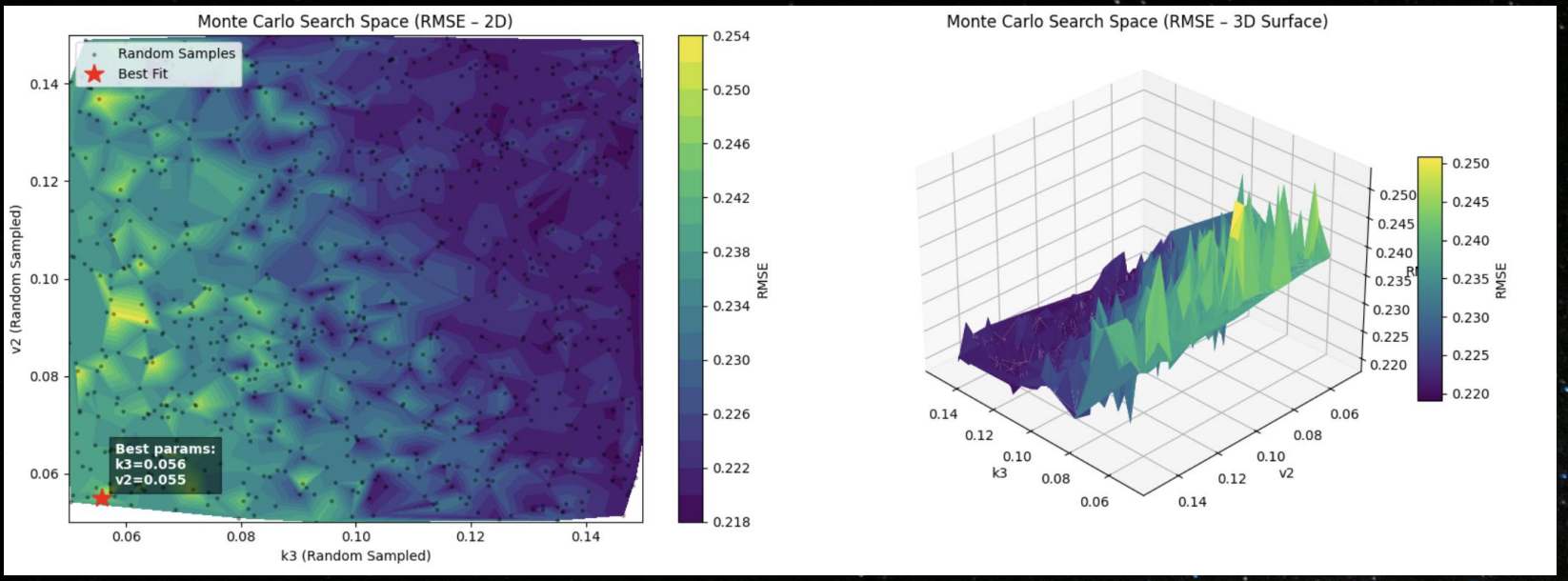


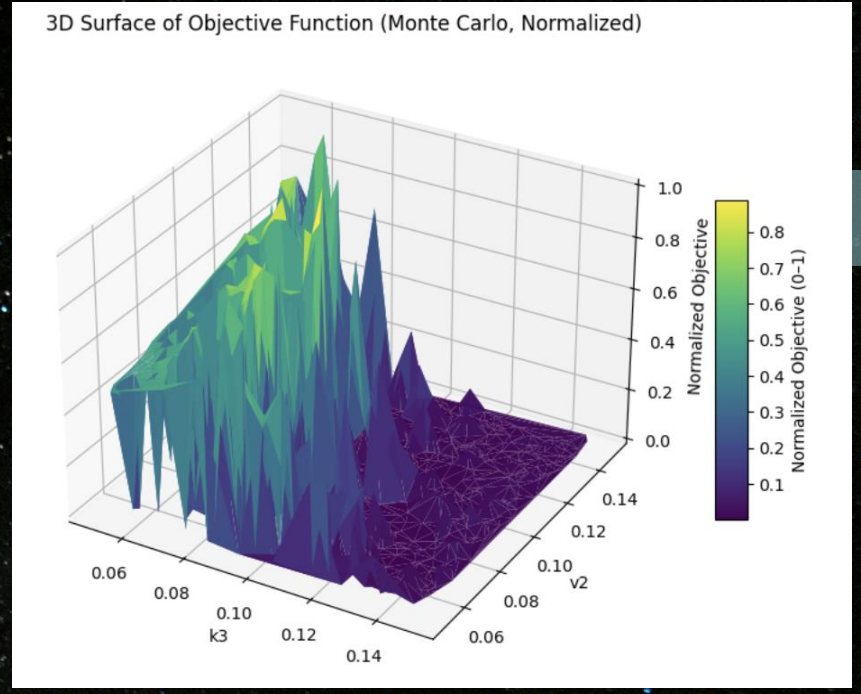


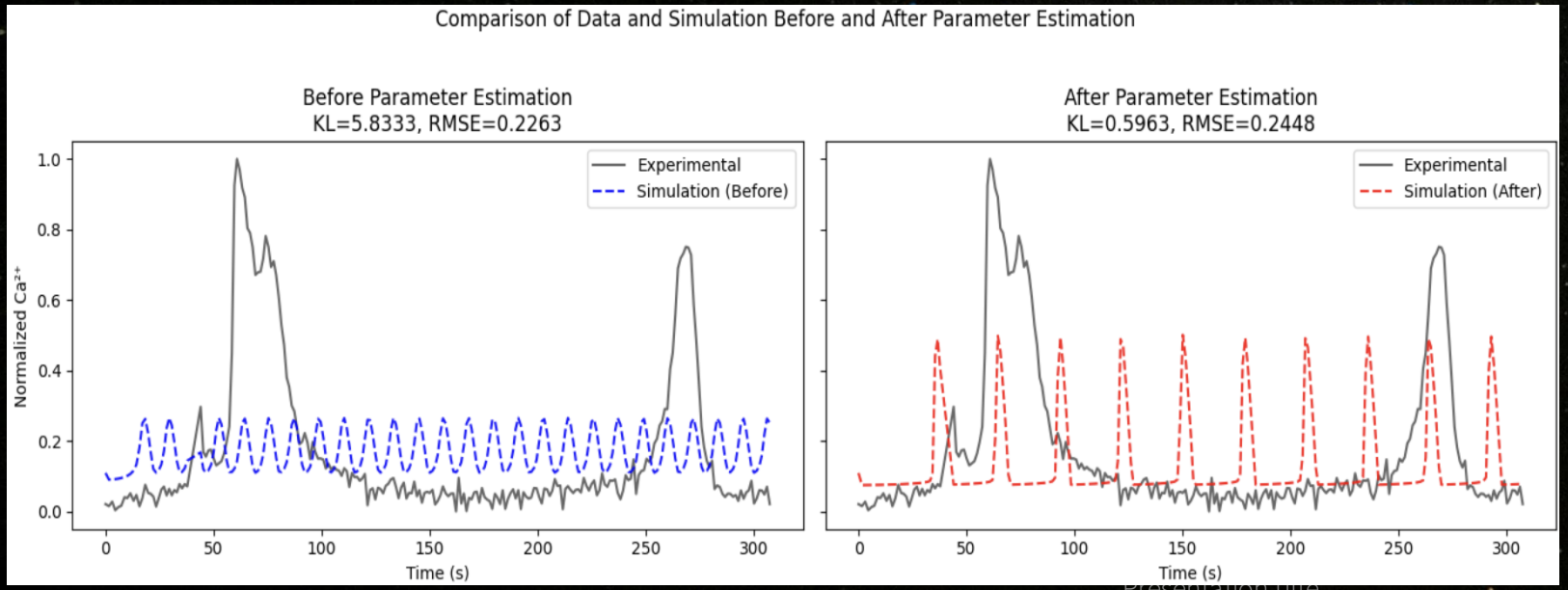


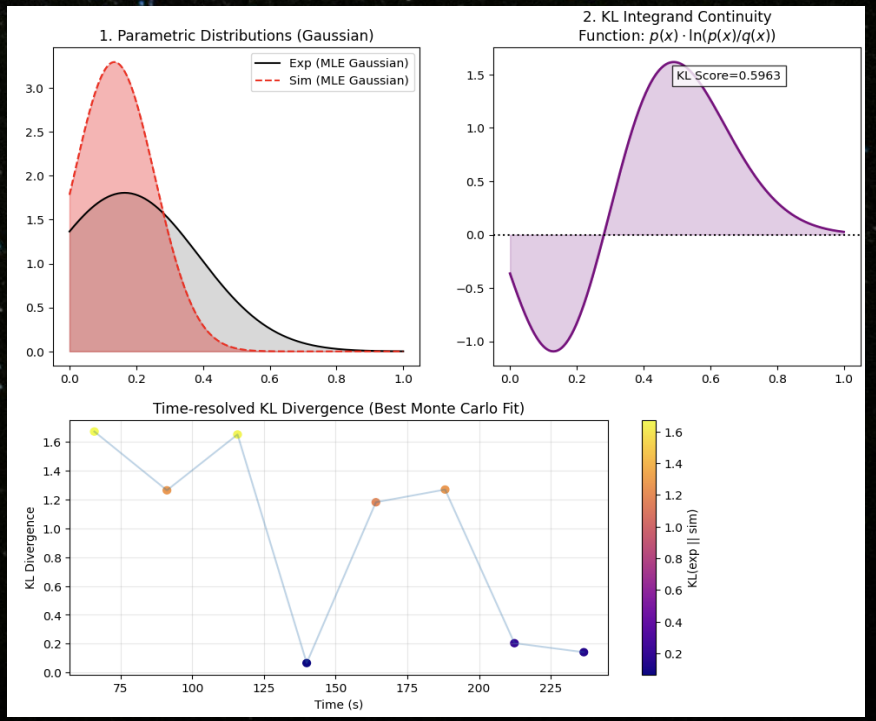


Column 3-harsha

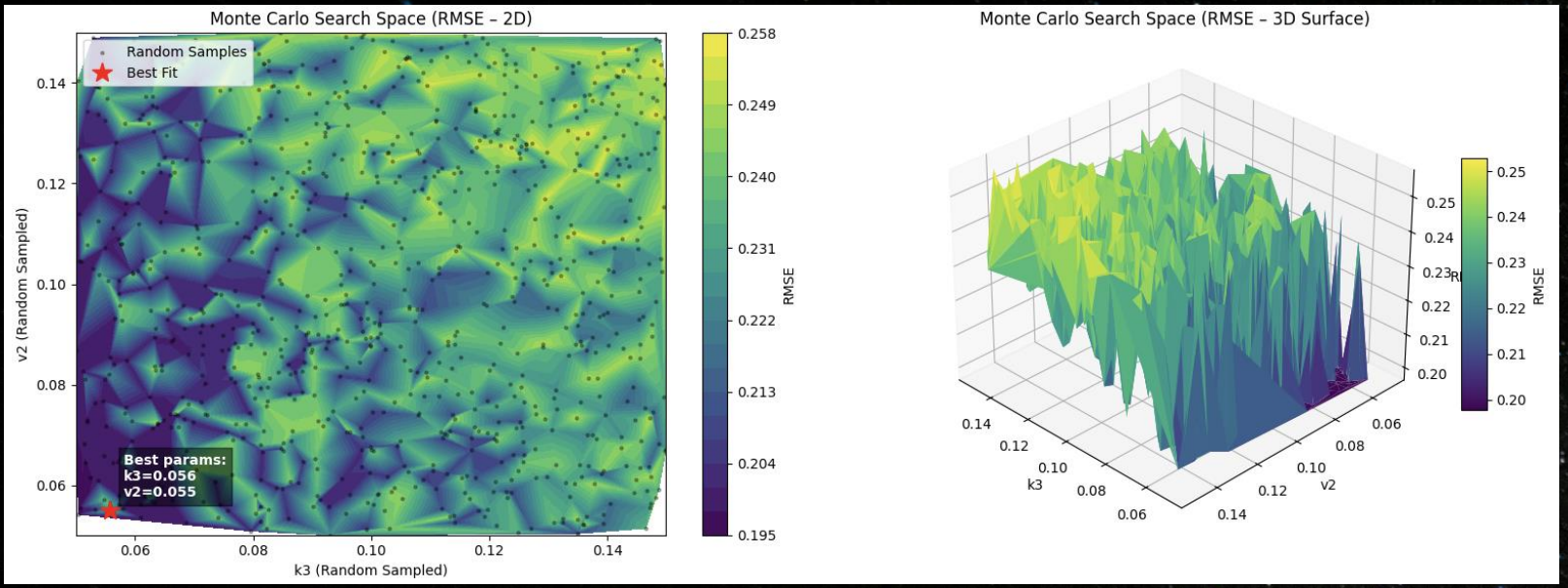


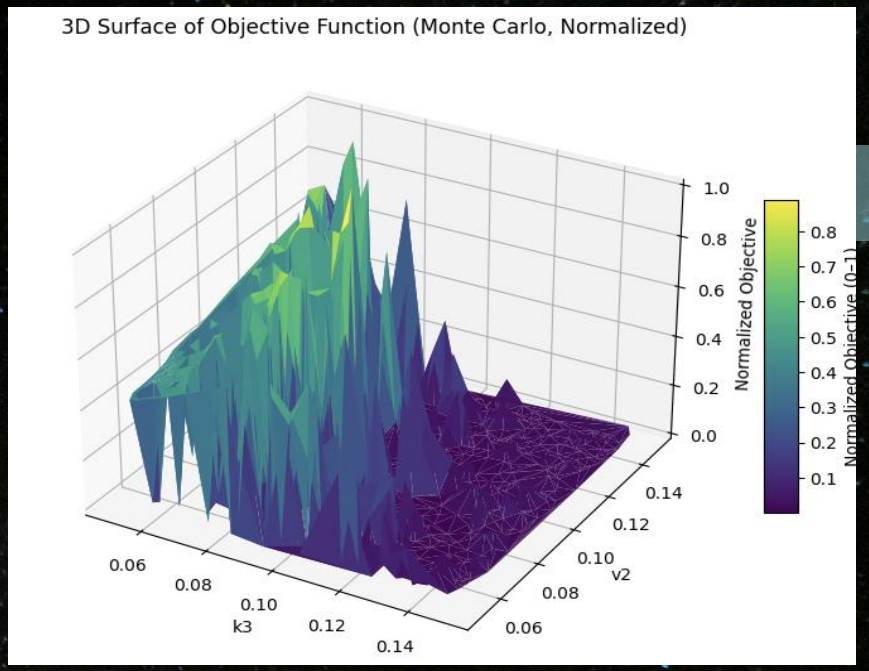


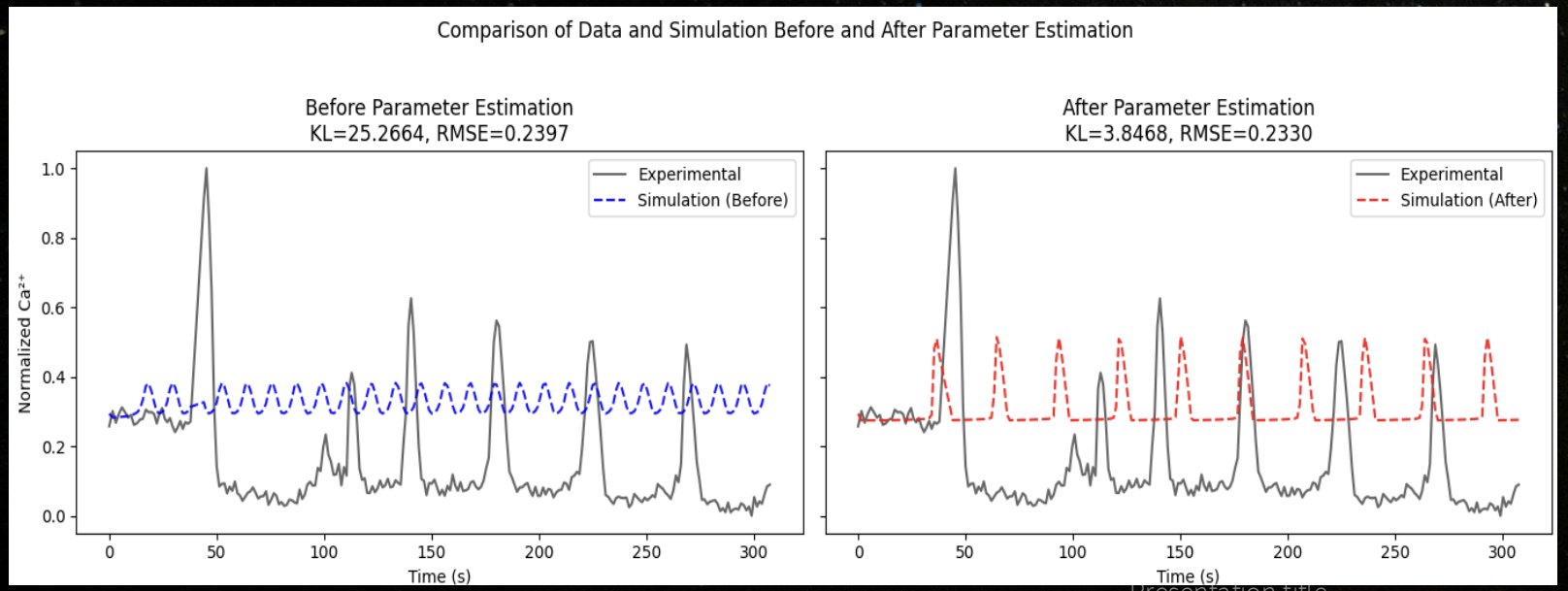


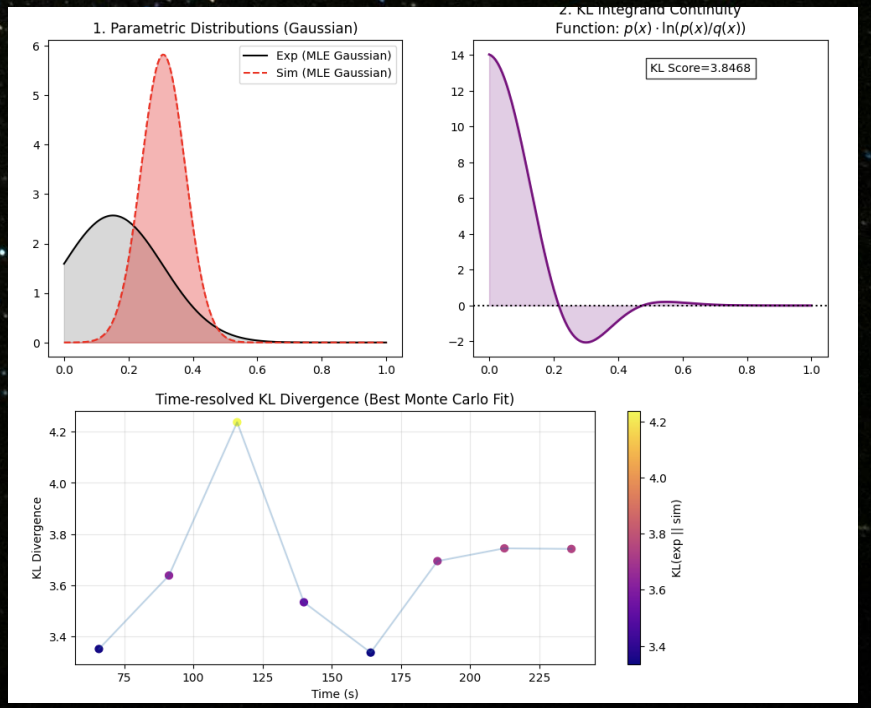


Column 4- VIVEK

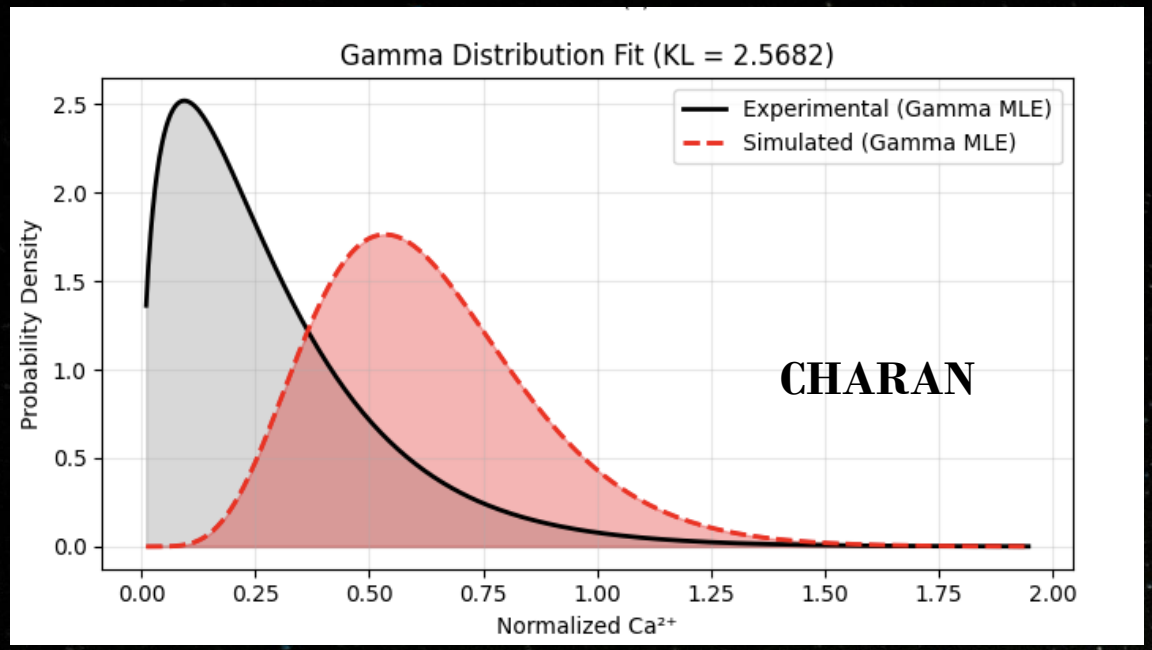


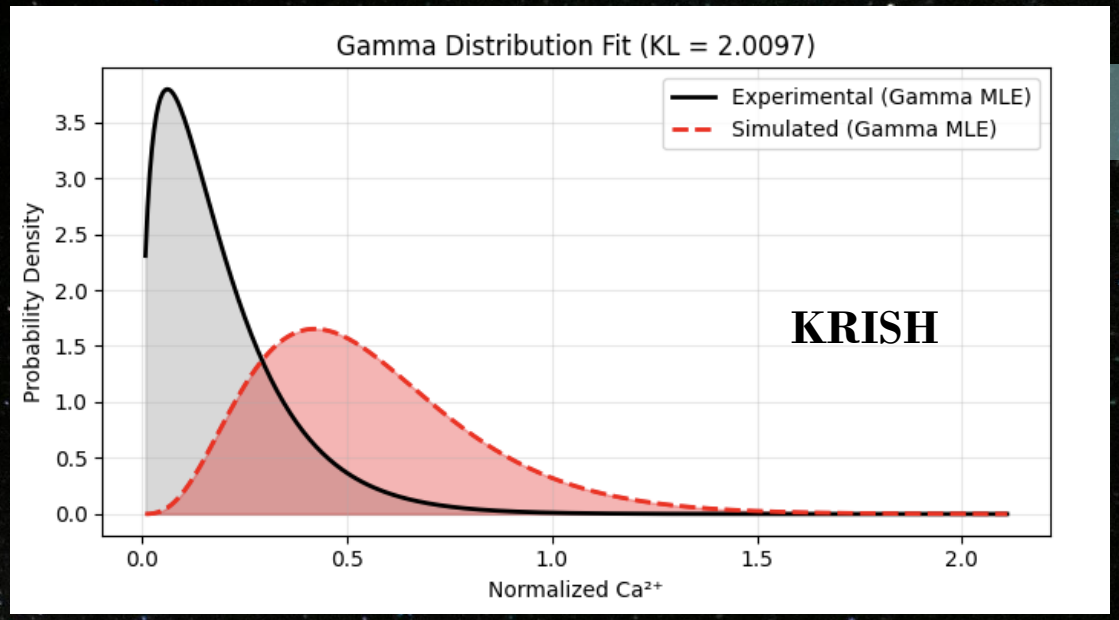


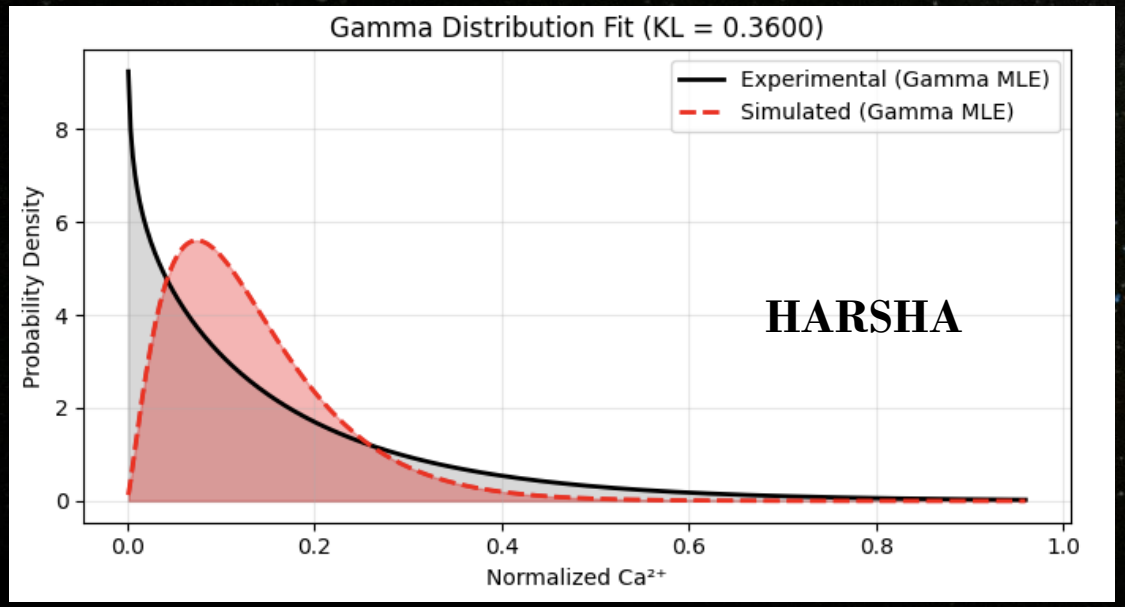


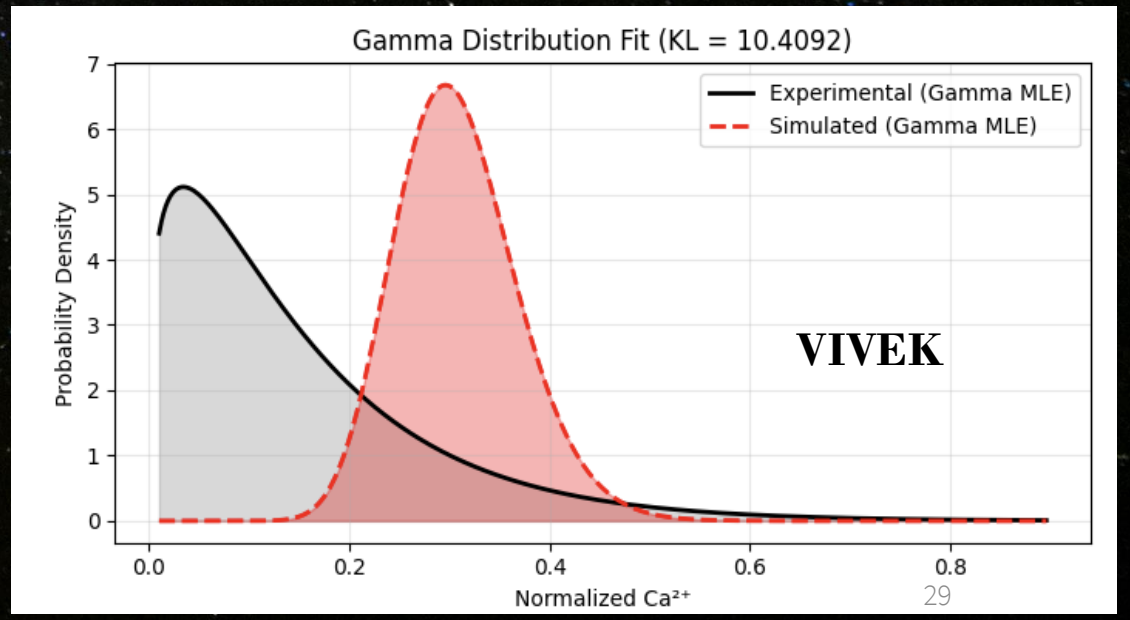


Gamma visualizations









Frequency Analysis (FFT Visualization) — Contributor: CHARAN

Core Purpose:

- **To verify that the optimized simulation not only matches the shape of the calcium trace but also reproduces the dominant oscillation frequency seen in the experimental data.**
- **FFT (Fast Fourier Transform) is used to extract frequency components from both signals.**

Preprocessing & Safety Checks:

- **Before applying FFT, the function ensures:**

1. NaN Handling:

- **If either experimental or simulated signals contain NaNs, they are filled via linear interpolation.**
- **This prevents FFT from failing.**

2. Constant Signal Check:
   - If a signal is flat or nearly constant, FFT becomes meaningless → warnings printed.
3. Sampling Interval (dt) Validation:
   - Computes dt from time vector.
   - If missing or invalid, defaults to dt = 0.1.
4. Length Alignment:
   - FFT requires equal-length arrays → shorter signal is padded with last sample.

FFT Computation:

1. Mean Removal

- Both signals are centered:
- $x' = x - mean(x)$
- This avoids energy at 0 Hz dominating the FFT.

2. Compute FFT:

- Using:
  - rfft() — real FFT
  - rfftfreq() — corresponding frequencies
- We obtain amplitude spectra:
  - $Y(f) = | FFT(x') |$

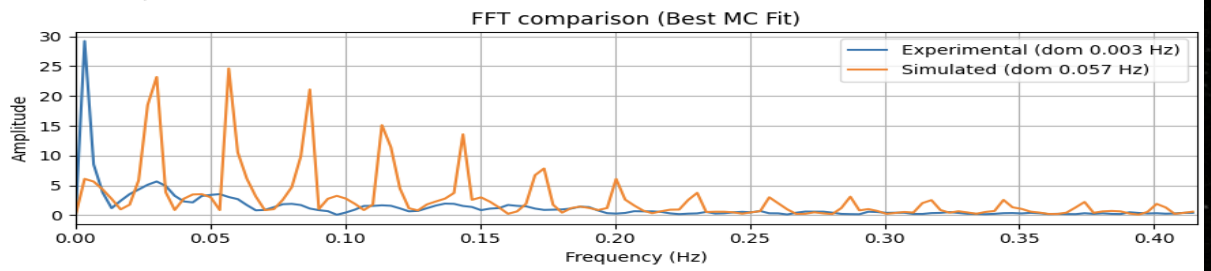
3. Dominant Frequency Extraction:

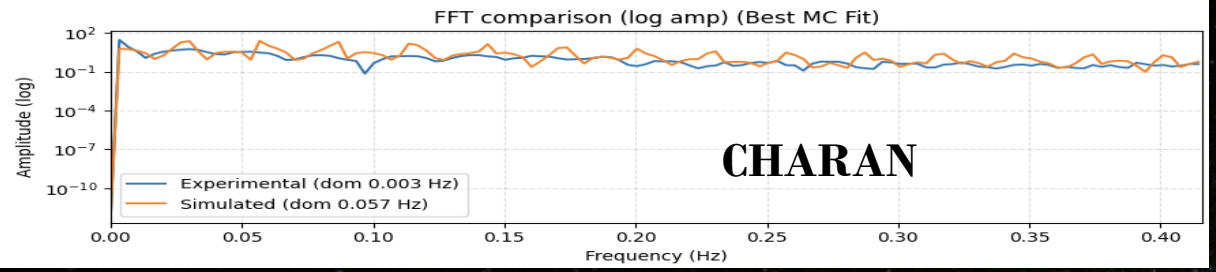
- A helper function finds:
  - $f_{dom} = \arg\max_{f>0} \quad Y(f)$
- Reported for experimental and simulated data.

Slide Takeaway:
Matching dominant frequencies validates oscillatory dynamics in the Li–Rinzel model.

FFT VISUALISATIONS

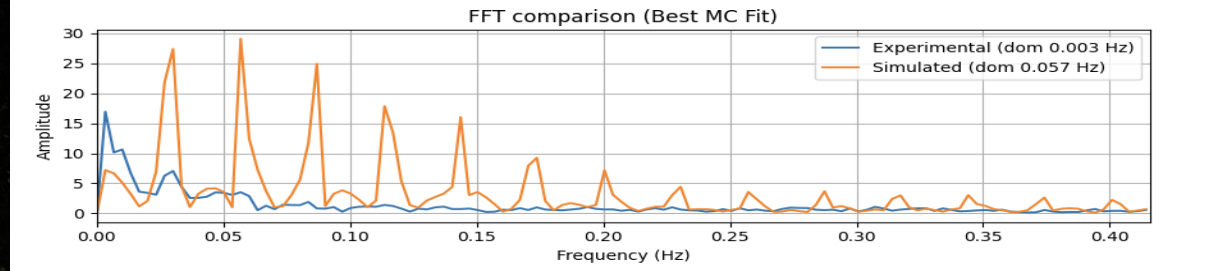
```
Debug (first 8 samples):
 exp_norm: [0.502877 0.511186 0.495133 0.474978 0.478155 0.458896 0.448009 0.437447]
 Ca_best : [0.608396 0.49447  0.49472  0.495091 0.495623 0.496259 0.497046 0.497884]
Dominant freq (experimental): 0.003341 Hz
Dominant freq (simulated)   : 0.056800 Hz
```

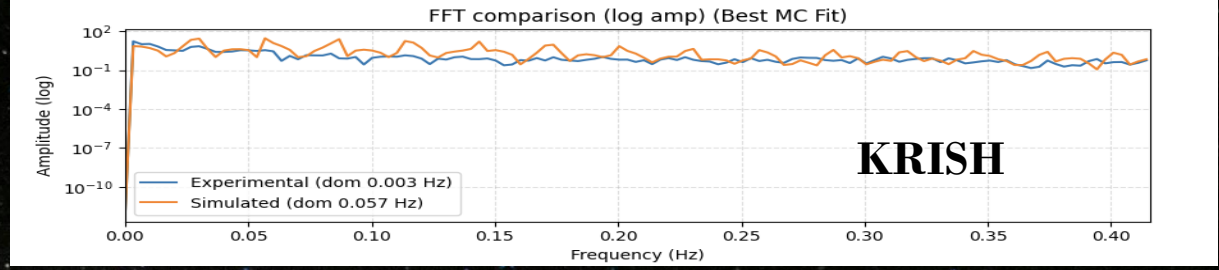




CHARAN

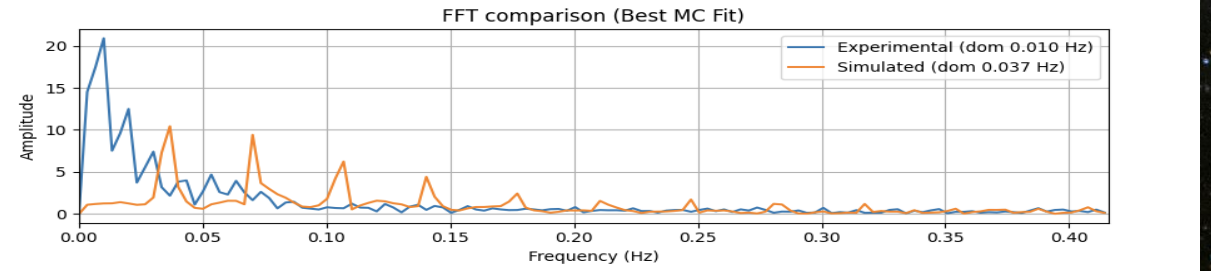
```
Debug (first 8 samples):
 exp_norm: [0.408961 0.420589 0.418428 0.360958 0.399797 0.398719 0.399285 0.387528]
 Ca_best : [0.529869 0.395359 0.395653 0.396092 0.396719 0.39747  0.3984   0.399389]
Dominant freq (experimental): 0.003341 Hz
Dominant freq (simulated)   : 0.056800 Hz
```

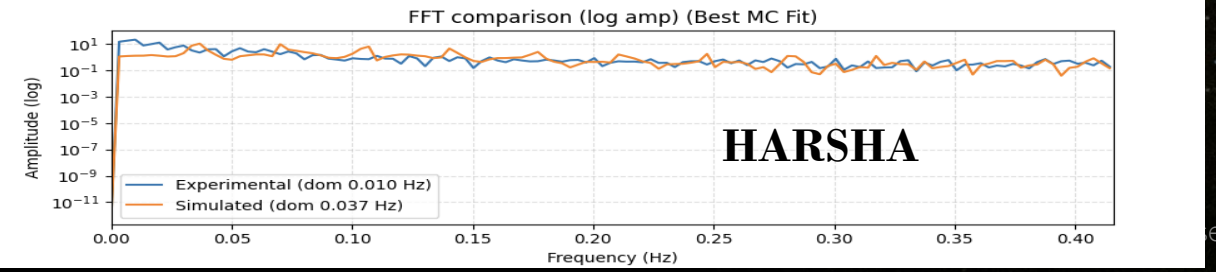




KRISH

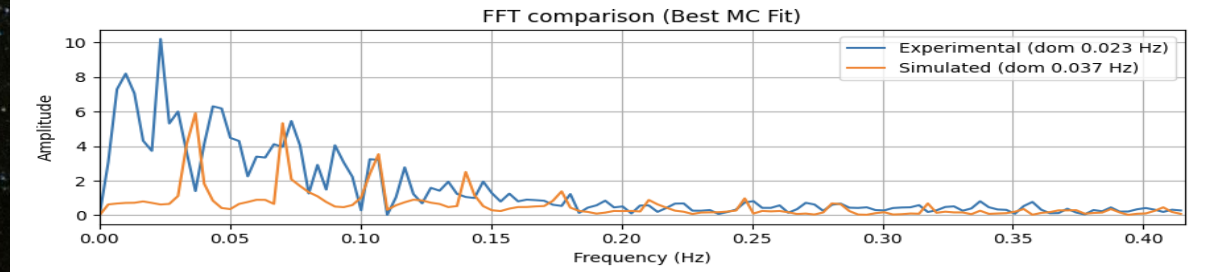
```
Debug (first 8 samples):
 exp_norm: [0.020508 0.014921 0.027287 0.003347 0.011887 0.01616  0.039891 0.037992]
 Ca_best : [0.108948 0.074481 0.074527 0.074595 0.074693 0.074812 0.074963 0.075127]
Dominant freq (experimental): 0.010023 Hz
Dominant freq (simulated)   : 0.036753 Hz
```

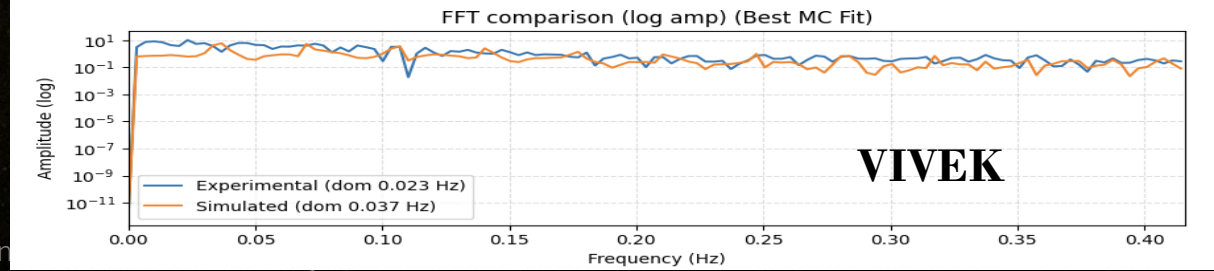




HARSHA

```
Debug (first 8 samples):
 exp_norm: [0.257475 0.301372 0.267647 0.293217 0.312049 0.298491 0.28248  0.289879]
 Ca_best : [0.293782 0.274279 0.274305 0.274344 0.274399 0.274466 0.274552 0.274645]
Dominant freq (experimental): 0.023388 Hz
Dominant freq (simulated)   : 0.036753 Hz
```





VIVEK

THANK YOU

GROUP-9